

Assembleia 01 — Plataforma Multiplayer

Data: 31/08/2026 **Método:** docs/METODO-PLANEJAMENTO.md **Conduzido por:** Orchestrator (orchestrator-mwrpg) **Status:** aguardando aprovação explícita do Tiago — nada implementado a partir daqui

Nota sobre o "projeto antigo"

O Tiago pediu para eu ler o projeto antigo de RPG de mesa dele que rodava no Discord antes de montar esta assembleia. Cloneiei e inspecionei o histórico completo de github.com/tiagobocchino/mwrpg.git (branch main e a branch solta `claude/db-migration-planning-javmZ`) e não há nenhum código, commit ou documento de um bot de Discord ali — o primeiro commit desse repositório já é o protótipo browser atual (React + Babel), e o histórico entre ele e os commits desta sessão é só um acidente de outro projeto ("Analytics Platform") que foi commitado e depois limpo. Isso ficou sinalizado pro Tiago em chat; ele ainda não respondeu onde o projeto de Discord realmente está. Esta assembleia parte do que **existe de fato e foi lido a fundo**: o protótipo browser (CLAUDE.md, Relatorio_Pesquisa_RPG.md, src/) — e trata "continuidade com o Discord" como uma pergunta em aberto, não como algo resolvido.

1. Baseline (munção pro debate, não proposta fechada)

Planejamento geral

Pivotar de protótipo single-player com mestre offline/localStorage (v0.1+v0.2, já em produção nesta sessão) para uma plataforma acessível por navegador (celular e computador) onde jogadores criam salas, criam personagem, decidem jogar solo ou em grupo, e o Mestre IA (+ NPCs) narra de verdade via Claude API — não mais o fallback offline.

Mapa de Implementação (fases candidatas)

- **Fase 0 (feito):** sistema de regras, design system, mestre offline + localStorage
- **Fase 1:** Mestre IA real via API (tira do modo offline) — ainda 1 jogador
- **Fase 2:** Contas + sala + personagem persistidos em nuvem
- **Fase 3:** Multiplayer de verdade — N jogadores na mesma sala, sincronizados
- **Fase 4:** NPCs mais ricos, conteúdo expandido, RAG (roadmap v0.4+ original)

Mapa de Estruturas (candidato)

```
/api/master → rota serverless que chama Claude (AI Master Engineer)
src/realtime.js → cliente de sincronização de sala (Realtime Multiplayer Engineer)
src/auth.js → sessão de usuário (Backend Engineer)
src/data.js → deixa de ser o único cenário fixo; vira catálogo
```

Mapa de Entidades (candidato)

User (conta) · Room (sala — código/convite, dono) · RoomMember (usuário × sala × personagem) · Character (ficha — sucessor do player fixo de data.js) · CampaignSession (histórico de turnos — sucessor do history/messages de storage.js) · TurnEvent (uma ação/narração — granularidade pra sincronizar ao vivo)

2. Consulta individual aos agentes

(Cada resposta abaixo veio de uma consulta separada, com o mesmo baseline acima, sem ver a resposta dos outros — ver a honestidade sobre o método em docs/METODO-PLANEJAMENTO.md.)

Realtime Multiplayer Engineer

Objetivos: decidir o mecanismo de tempo real sem repetir o erro do PushProcessos (infra que hiberna mata sessão ao vivo). **Planejamento:** todo candidato sério para isto (Supabase Realtime, Pusher, Ably, Cloudflare Durable Objects/PartyKit) é um **serviço gerenciado cobrado por uso/conexão, não um processo próprio sempre-ligado que dorme por inatividade** — essa é a diferença estrutural que importa, não hibernam do jeito que o Render free Web Service hiberna. Isso resolve a preocupação levantada pelo Tiago de raiz, independente de qual desses provedores for escolhido. **Processos:** avaliar Supabase Realtime primeiro por já ser o provedor mais usado no ecossistema de projetos do Tiago (menos peça nova pra aprender) — Postgres Changes pra estado de sala, Presence pra "quem está online", Broadcast pra eventos efêmeros (dado rolando). **Tarefas:** confirmar limites exatos do tier free no dashboard antes de comprometer (não assumir número de memória); prototipar reconexão (jogador cai e volta sem duplicar mensagem). **Divergência que registro:** existe uma alternativa mais barata ainda — multiplayer **assíncrono por turno** (como jogo por correspondência: a sala atualiza quando a página recarrega/consulta, não empurra em tempo real) — muito mais simples e sem custo de conexão persistente, mas não é "jogar junto ao vivo" no sentido literal que o Tiago descreveu. Marco isso como eixo de variação pros finalistas.

AI Master Engineer

Objetivos: tirar o mestre do modo offline sem custo surpresa por sessão. **Planejamento:** consulte a tabela de preços vigente (skill claude-api deste ambiente, não memória): Haiku 4.5 = \$1/\$5 por milhão de tokens (entrada/saída); Sonnet 5 = \$2/\$10. Estimativa pra uma sessão de ~20 turnos, contexto crescendo (~3.000 tokens de entrada médios por turno com histórico acumulado, ~300 tokens de narração de saída por turno): **~60k tokens de entrada + ~6k de saída por sessão → Haiku ≈ US\$0,09/sessão, Sonnet ≈ US\$0,18/sessão**. Com prompt caching (o SYSTEM_PROMPT e o início do histórico são estáveis dentro de uma campanha — candidatos naturais a cache), o custo real de entrada cai ainda mais ao longo da sessão. Pra 100 sessões de teste: **Haiku ≈ US\$9, Sonnet ≈ US\$18** — trivial mesmo no cenário mais caro. **Processos:** manter o mestre narrando por todos os NPCs numa única chamada JSON (já é como master.js funciona hoje) em vez de uma chamada por NPC — evita multiplicar o custo por 3-4x sem necessidade. **Tarefas:** portar SYSTEM_PROMPT pra rota serverless; manter contrato JSON idêntico; decidir streaming (typewriter) como melhoria depois do MVP, não bloqueante. **Divergência que registro:** Haiku é suficiente e muito mais barato, mas a qualidade de narração ("sábio, consciente, sincero, direto") é o diferencial do produto — Sonnet narra melhor. Marco como eixo de variação.

Backend Engineer

Objetivos: persistência de sala/personagem/campanha em nuvem, substituindo o `localStorage` da v0.2 sem perder o que ele já resolve (retomar campanha). **Planejamento:** Supabase (Auth + Postgres + RLS) é o candidato natural — o próprio CLAUDE.md original já cogitava isso pra v0.4/v0.5, e é o mesmo provedor que o Realtime Multiplayer Engineer recomenda, o que reduz peça móvel (uma conta, um provedor, RLS cobrindo tanto dado quanto canal). Free tier já documentado no próprio CLAUDE.md original: 500MB DB, 5GB bandwidth — folgado pro volume de texto de uma fase de teste. **Processos:** RLS por `room_id/user_id` — jogador nunca lê ficha ou sala que não é sua. **Tarefas:** schema mínimo (User/Room/RoomMember/Character/ CampaignSession); migrar o formato de `storage.js` pro banco sem perder compatibilidade de campanhas já salvas localmente. **Objeção que registro:** qualquer finalista que abra mão de persistência/contas pra "simplificar" está jogando fora uma feature que o Tiago **já tem e já validou** nesta sessão (v0.2, retomar campanha) — regressão, não simplificação.

Infra Engineer

Objetivos: manter tudo rodando sem custo relevante e sem hibernar em sessão ao vivo. **Planejamento:** ponto de partida já decidido nesta sessão — Vercel Hobby pro frontend estático (não hiberna, já configurado). Pra `/api/master`, Vercel Functions/Edge Functions têm o mesmo perfil estrutural que o Realtime Engineer descreveu: cobradas por invocação, não um processo sempre-ligado que dorme. **O padrão a evitar em qualquer peça nova da stack é "um único free Web Service sempre-ligado" (estilo Render free)** — isso vale tanto pra API quanto pro realtime. **Processos:** confirmar limite de duração de execução de function serverless compatível com o tempo de resposta do mestre IA (streaming ajuda a não estourar timeout em respostas longas). **Tarefas:** variáveis de ambiente (ANTHROPIC_API_KEY, chaves do Supabase) só em `env var` do provedor, nunca em código. **Objeção que registro:** introduzir build step (Vite) e infra de CI antes de saber se alguém vai jogar de verdade é gastar esforço de infra num produto ainda não validado — prematuro pro MVP, correto mais adiante.

Game System Designer

Objetivos: o sistema D6 das Três Letras funciona solo; adaptar pra grupo sem reescrever a fórmula. **Planejamento:** manter a regra de que quem escolheu a ação é quem rola (já é assim hoje); em grupo, o mestre narra pra sala inteira o resultado de cada rolagem individual — não precisa de mecanismo novo de "iniciativa formal" pro MVP, isso pode esperar (roadmap v0.6, combate tático, já existe como item futuro no CLAUDE.md original). **Tarefas:** fluxo de criação de personagem é um buraco real — hoje o player é fixo em `data.js`; qualquer plataforma multiusuário precisa disso desde o primeiro dia, independente de qual finalista for escolhido. **Sem objeção de desclassificação** a nenhum finalista — esse ponto vale pra todos igualmente.

Narrative Writer

Objetivos: manter o tom "sábio, consciente, sincero, direto" mesmo narrando pra múltiplos jogadores. **Planejamento:** o cenário "A Coroa Enterrada de Ys" já é robusto o bastante pra suportar múltiplas salas jogando em paralelo sem reescrever nada — não é bloqueio de MVP. Pra fase de teste, **um cenário bem polido supera vários rasos. Divergência que registro, com peso:** dado que a tabela de custo do AI Master Engineer mostra a diferença entre Haiku e Sonnet em **centavos de dólar por sessão**, e que a narrativa é o diferencial de produto declarado no próprio CLAUDE.md (pilar 1: "UI fora dos padrões" + tom literário), **defendo Sonnet como modelo padrão**, não Haiku — o custo de "economizar" é pequeno demais pra justificar arriscar um mestre mais raso.

Frontend Engineer

Objetivos: lobby, sala, presença — sem quebrar o design system "Manuscrito Vivo" nem a prioridade mobile-first. **Planejamento:** as telas novas (entrar/criar sala, lista de presença) são extensões do design system existente, não redesenho. **Divergência que registro:** o zero-build (Babel standalone) é ótimo pro protótipo atual, mas autenticação real + assinatura de canais de realtime tende a empurrar a complexidade de estado — ainda assim, **recomendo não migrar pra Vite agora:** validar com usuários reais primeiro, revisitar se a dor aparecer. Concordo com o Infra Engineer que isso seria esforço prematuro. **Objeção que registro:** um finalista que corte contas/persistência pra "simplificar" (visão de MVP mínimo) não é de fato mais simples pro frontend — ainda precisa de fluxo de sala, só que sem poder confiar em nada salvo, o que na prática exige mais tratamento de estado efêmero, não menos.

Code QA Engineer

Objetivos: garantir isolamento entre salas/jogadores e nenhum segredo exposto, seja qual for o finalista. **Planejamento:** qualquer fluxo de "código de sala" (convite) precisa ser token longo e aleatório, nunca um código curto adivinhável — um código de sala curto e sequencial é uma superfície de invasão de sala alheia, e isso vale mesmo na opção "mais simples" avaliada. **Checklist que aplico a qualquer finalista escolhido:** contrato JSON do mestre intacto, sem const styles global, sem segredo em código, isolamento por room_id/user_id real (não só por convenção de UI). **Objeção que registro:** um finalista "sem conta nenhuma, sala aberta por código curto" não é mais simples de proteger — é mais difícil, porque não há usuário dono verificável pra revogar acesso.

Test Engineer

Objetivos: garantir que "passou no code review" e "funciona de verdade, jogado" não sejam coisas diferentes. **Planejamento:** pra multiplayer, teste manual real significa abrir múltiplas abas/perfis de navegador simulando jogadores diferentes na mesma sala e jogar uma sessão inteira entre elas — não só uma aba. **Tarefas:** matriz mínima — sessão solo encadeada, sala com 2 jogadores, queda e reconexão de um deles, viewport mobile 375px. **Objeção que registro:** quanto mais peças novas de infra (build step, múltiplos provedores) um finalista introduz de uma vez, maior a superfície de teste antes do primeiro usuário real — prefiro o finalista que adiciona uma peça de cada vez.

3. Síntese — convergência e divergência

Convergência (vira parte fixa de qualquer plano):

- Nenhuma peça nova de infra pode ser um único "free Web Service sempre-ligado" que hiberna — Realtime, Infra e Backend convergem nisso independentemente.
- Supabase como provedor único de Auth+DB+Realtime reduz peça móvel — Backend e Realtime convergem.
- Mestre narra por todos os NPCs numa chamada só (não uma por NPC) — AI Master e Game System Designer convergem.
- D6 das Três Letras e contrato JSON do mestre não mudam de fórmula — Game System Designer, AI Master e Code QA convergem.
- Fluxo de criação de personagem é gap real e obrigatório em qualquer finalista — Game System Designer isolado, mas ninguém discorda.

- Convite de sala por token longo, nunca código curto adivinhável — Code QA, com Backend concordando.
- Teste manual precisa simular múltiplos jogadores em abas separadas — Test Engineer, Code QA concorda.
- Não migrar pra build step (Vite) ainda — Frontend e Infra convergem.

Divergência real (vira eixo de variação nos finalistas):

- **Eixo A — sincronização:** tempo real de verdade (push via Realtime) vs. multiplayer assíncrono por turno (mais barato e simples, mas não é "jogar junto ao vivo" literal).
 - **Eixo B — modelo do mestre:** Haiku por padrão (barato) vs. Sonnet por padrão (narração mais rica, ainda barato em termos absolutos).
 - **Eixo C — persistência/contas:** conta real + nuvem desde o dia 1 (mantém a v0.2 já entregue) vs. cortar pra validar mais rápido (Backend e Frontend já registraram objeção a isso).
-

4. Cinco finalistas

Finalista 1 — "MVP Assíncrono"

Multiplayer por turno sem push em tempo real (sala atualiza ao recarregar/consultar); Supabase só para persistência (sem canal Realtime); Haiku por padrão; sem build step; contas via magic link. Mais barato e simples de operar, mas não entrega "jogar junto ao vivo".

Finalista 2 — "Equilíbrio"

Supabase Realtime (Presence + Broadcast) para sincronização ao vivo + Postgres para persistência; Haiku por padrão, com opção de trocar por sala; convite por token longo; contas via magic link; sem build step por enquanto. Combina a maior parte dos pontos convergentes.

Finalista 3 — "Aposta em Qualidade Narrativa"

Igual ao Finalista 2 em infra, mas Sonnet 5 como modelo padrão do mestre, aceitando ~2x o custo (ainda assim centavos por sessão) em troca de narração mais rica — prioriza o pilar de produto "tom literário diferenciado".

Finalista 4 — "App-Ready desde já"

Igual ao Finalista 2 em infra de dados/realtime, mas já migra pra Vite + build step agora, antecipando o item de roadmap "app nativo futuro"; mais esforço de engenharia inicial.

Finalista 5 — "Mínimo Absoluto pra Validar"

Sem conta de usuário (sala anônima por link), sem persistência em nuvem (mantém só o `localStorage` por dispositivo já existente da v0.2), sincronização por polling simples em vez de Realtime; um único cenário; Haiku. Objetivo: validar se alguém quer jogar multiplayer *nesta semana* antes de investir em infra.

5. Votação

Agente	Voto	Justificativa / objeção
Realtime Multiplayer Engineer	#2	Tiago pediu "jogar juntas" — tempo real de verdade, não assíncrono (desclassifica #1 como plano principal, não como spike).
AI Master Engineer	#2	Haiku é suficiente e extensível por sala; gastar 2x por padrão (#3) é prematuro antes de medir se usuários notam a diferença.
Backend Engineer	#2	Supabase único provedor. Objeta a #5: cortar persistência é regredir uma feature já entregue (v0.2).
Infra Engineer	#2	Estrutura sem hibernação, custo baixo confirmado. Desclassifica #4 para o MVP: build step antes de validar produto é esforço fora de ordem.
Game System Designer	#2	Indiferente ao eixo de infra; reforça que criação de personagem é obrigatória em qualquer um.
Narrative Writer	#3 (dissenso)	Diferença de custo é centavos; narração é o diferencial do produto — vale o Sonnet por padrão.
Frontend Engineer	#2	Concorda com Infra sobre #4; nota que #5 não é de fato mais simples pro frontend (mais estado efêmero pra tratar, não menos).
Code QA Engineer	#2	Reforça token de convite longo. Objeta a #5: sala anônima sem dono verificável é mais difícil de proteger, não mais fácil.

Test Engineer	#2	Menos peças novas de uma vez = menos superfície de teste antes do primeiro usuário real; desclassifica #4 pelo mesmo motivo que Infra.
---------------	----	--

Resultado: Finalista 2 ("Equilíbrio") vence 8 a 1.

6. Vencedor — com as mitigações da minoria incorporadas

Finalista 2, "Equilíbrio", com três ajustes baratos vindos das objeções que não venceram mas têm peso técnico real:

- 1 **Do dissenso do Narrative Writer** — modelo do mestre configurável por sala desde o primeiro dia (Haiku por padrão, Sonnet como opção), em vez de fixo. Resolve o trade-off sem comprometer o padrão barato.
- 2 **Da objeção do Code QA Engineer** — convite de sala sempre por token longo/aleatório, nunca código curto — já estava no finalista, reforçado como requisito não-negociável de segurança, não só de UX.
- 3 **Do ponto cross-cutting do Game System Designer** — fluxo de criação de personagem entra no escopo do MVP, não é adiável (hoje o personagem é fixo em `data.js`; qualquer plataforma multiusuário precisa disso pra ter um segundo jogador jogável).

O plano, resumido

- **Sincronização:** Supabase Realtime (Postgres Changes + Presence + Broadcast) — nenhuma peça de infra hiberna em inatividade.
- **Persistência:** Supabase Postgres + RLS por sala/usuário — sucessor do `localStorage` da v0.2, não substituição que perde a feature.
- **Mestre IA:** rota serverless (Vercel Functions) chamando Claude API — Haiku 4.5 por padrão (~US\$0,09/sessão de 20 turnos), Sonnet 5 disponível por sala (~US\$0,18/sessão) — contrato JSON de `master.js` mantido.
- **Frontend:** continua zero-build (Babel standalone) por enquanto; revisar depois da fase de teste se a complexidade de estado doer.
- **Escopo mínimo pra usuários de teste reais:** contas (magic link) + criação de personagem + criar/entrar em sala por convite (token longo) + jogar uma sessão multiplayer completa do cenário "A Coroa Enterrada de Ys" do início ao fim, com narração real (não offline).

Em aberto — não decidido nesta assembleia

- **Onde está o projeto de Discord antigo** que o Tiago mencionou — sem essa resposta, não dá pra saber se há conteúdo/regra/decisão de design específica de lá que deveria influenciar este plano.